

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	Putta Yasasvi
ROLL NO	AV.SC.U4CSE25235
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5-7
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	7-8
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	8-10
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	10-11
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index	12-15

	5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10)Search data in SLL 11)Display or Traverse 12)Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10)Exit	16-19
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	19-22
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	23-26
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion	26-29

	3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	29-33
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	33-36
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	36-37

WEEK – 1

PROGRAM-1(A):

AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!".

CODE:

```
def countdown(n):
    if n == 0:
        print("Launch!")
        return
    print(n)
    countdown(n - 1)
n = int(input("Enter countdown number: "))
countdown(n)
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/C
Enter countdown number: 5
5
4
3
2
1
Launch!
>>>
```

PROGRAM-1(B):

AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years.

CODE:

```
def power(p, n):
    if n == 0:
        return 1
    return p * power(p, n - 1)
p = int(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))
print("Power =", power(p, n))
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/
Enter principal growth factor: 2
Enter number of years: 5
Power = 32
>>>
```

PROGRAM-1(C):

AIM: A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

```
def search(list, key, index):
    if index == len(list):
        return False
    if list[index] == key:
        return True
    return search(list, key, index + 1)
emp_ids = [101, 102, 103, 104, 105]
key = int(input("Enter employee ID to search: "))
if search(emp_ids, key, 0):
    print("Employee ID found")
else:
    print("Employee ID not found")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/
Enter employee ID to search: 104
Employee ID found
>>>
```

PROGRAM-1(D):

AIM: In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
num = int(input("Enter a number: "))
print("Factorial =", factorial(num))
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/
Enter a number: 7
Factorial = 5040
>>>
```

PROGRAM-1(E):

AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

CODE:

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)
terms = int(input("Enter number of terms: "))
print("Fibonacci Series:")
for i in range(terms):
    print(fibonacci(i), end=" ")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/
Enter number of terms: 8
Fibonacci Series:
0 1 1 2 3 5 8 13
>>>
```

WEEK – 2

PROGRAM-2(A):

AIM: Write a python program to implement linear search.

CODE:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
arr = list(map(int, input("Enter array elements: ").split()))
key = int(input("Enter element to search: "))
result = linear_search(arr, key)
if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/
Enter array elements: 10 32 11 12 12
Enter element to search: 7
Element not found
>>>
```

PROGRAM-2(B):

AIM: Write a python program to implement binary search with and without sorted array as an input.

CODE:

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
arr = list(map(int, input("Enter array elements: ").split()))
arr.sort()
print("Sorted array:", arr)
key = int(input("Enter element to search: "))
result = binary_search(arr, key)
if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/
Enter array elements: 12 3 2 1 0
Sorted array: [0, 1, 2, 3, 12]
Enter element to search: 2
Element found at index 2
>>>
```

WEEK – 3

PROGRAM-3(A):

AIM: Write a python program to implement bubble sort.

CODE:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
n = int(input("Enter no.of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
bubble_sort(arr)
print("Sorted array using Bubble Sort:", arr)
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive
Enter no.of elements: 5
Enter the elements:
10
8
3
5
4
Sorted array using Bubble Sort: [3, 4, 5, 8, 10]
>>>
```

PROGRAM-3(B):

AIM: Write a python program to implement insertion sort.

CODE:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
n = int(input("Enter no.of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
insertion_sort(arr)
print("Sorted array using Insertion Sort:", arr)
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/I
Enter no.of elements: 5
Enter the elements:
3
2
1
4
5
Sorted array using Insertion Sort: [1, 2, 3, 4, 5]
>>>
```

PROGRAM-3(C):

AIM: Write a python program to implement selection sort.

CODE:

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
n = int(input("Enter no.of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
selection_sort(arr)
print("Sorted array using Selection Sort:", arr)
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/Deskt
Enter no.of elements: 5
Enter the elements:
17
12
11
5
2
Sorted array using Selection Sort: [2, 5, 11, 12, 17]
>>>
```

WEEK – 4

PROGRAM-4(A):

AIM: Write a python program to implement quick sort.

CODE:

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
n = int(input("Enter no.of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array using Quick Sort:", arr)
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/De
Enter no.of elements: 5
Enter the elements:
90
89
458
232
22
Sorted array using Quick Sort: [22, 89, 90, 232, 458]
>>>
```

PROGRAM-4(B):

AIM: Write a python program to implement merge sort.

CODE:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]
        merge_sort(left_half)
        merge_sort(right_half)
        i = j = k = 0
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1
n = int(input("Enter no.of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
merge_sort(arr)
print("Sorted array using Merge Sort:", arr)
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDriv
Enter no.of elements: 5
Enter the elements:
17
11
12
21
43
Sorted array using Merge Sort: [11, 12, 17, 21, 43]
>>>
```

WEEK – 5

PROGRAM-5(A):

AIM: Write a python program to implement a Singly Linked List with the below operations.

Create Linked List, Insert Beginning, Insert End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes,

Sort the data of SLL, Search data in SLL, Display or Traverse and Exit

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None
    def create(self):
        n = int(input(("Enter number of elements:")))
        for i in range(n):
            x = int(input(("Enter value:")))
            new = Node(x)
            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new
    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new
    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new
    def insert_index(self, index, data):
        if index == 0:
            self.insert_begin(data)
            return
        new = Node(data)
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid Index")
                return
            temp = temp.next
        new.next = temp.next
        temp.next = new
```

```
    def deleteAtBeg(self):
        if self.head is None:
            print("No Data to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted Value = ",temp.data)
    def deleteAtEnd(self):
        if self.head is None:
            print("No Data to delete")
        elif self.head.next is None:
            self.head = None
        else:
            temp = self.head
            temp1 = temp
            while temp.next:
                temp1 = temp
                temp = temp.next
            temp1.next = None
    def delete(self, value):
        if self.head is None:
            print("No Data to delete")
        else:
            temp = self.head
            if temp and temp.data == value:
                self.head = temp.next
                print("Value deleted")
                return
            while temp.next and temp.next.data != value:
                temp = temp.next
            if temp.next is None:
                print("Value not present")
            else:
                temp.next = temp.next.next
                print("Value deleted")
    def display(self):
        if self.head is None:
            print("No Data")
        else:
            temp = self.head
            while temp:
                print(temp.data, end = " -> ")
                temp = temp.next
            print("None")
```

```
    def count(self):
        if self.head is None:
            print("No Linked List")
        else:
            c = 0
            temp = self.head
            while temp:
                c += 1
                temp = temp.next
            print(f"Number of nodes = {c}")
    def search(self,target):
        if self.head is None:
            print("Cannot perform search")
            return
        temp = self.head
        while temp:
            if temp.data == target:
                print(f"{target} is found")
                return
            temp = temp.next
        print(f"{target} not found")
    def sort(self):
        if self.head is None:
            print("Cannot perform sorting")
            return
        i = self.head
        while i:
            j = i.next
            while j:
                if i.data > j.data:
                    i.data,j.data = j.data,i.data
                j = j.next
            i = i.next
        print("Sorting Completed")
SLL = SinglyLinkedList()
while True:
    print("\n1.Create Linked List")
    print("2.Insert Beginning")
    print("3.Insert End")
    print("4.Insert at Index")
    print("5.Delete by Value")
    print("6.Delete First Node")
    print("7.Delete Last Node")
    print("8.Count Number of Nodes")
    print("9.Searching")
    print("10.Sorting")
```

```
print("11.Display/Traverse")
print("12.Exit")
ch = int(input("Choice: "))
if ch == 1:
    SLL.create()
if ch == 2:
    x = int(input("Value: "))
    SLL.insert_begin(x)
elif ch == 3:
    x = int(input("Value: "))
    SLL.insert_end(x)
elif ch == 4:
    idx = int(input("Index: "))
    x = int(input("Value: "))
    SLL.insert_index(idx, x)
elif ch == 5:
    x = int(input("Delete value: "))
    SLL.delete(x)
elif ch == 6:
    SLL.deleteAtBeg()
elif ch == 7:
    SLL.deleteAtEnd()
elif ch == 8:
    SLL.count()
elif ch == 9:
    target = int(input("Enter value to search:"))
    SLL.search(target)
elif ch == 10:
    SLL.sort()
elif ch == 11:
    SLL.display()
elif ch == 12:
    break
else:
    print("Invalid option")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/D

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Searching
10.Sorting
11.Display/Traverse
12.Exit
Choice: 1
Enter number of elements:3
Enter value:10
Enter value:20
Enter value:30
Invalid option

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Searching
10.Sorting
11.Display/Traverse
12.Exit
Choice: 4
Index: 1
Value: 15
```

```
1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Searching
10.Sorting
11.Display/Traverse
12.Exit
Choice: 5
Delete value: 15
Value deleted

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Searching
10.Sorting
11.Display/Traverse
12.Exit
Choice: 8
Number of nodes = 3

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Searching
10.Sorting
11.Display/Traverse
12.Exit
Choice: 9
Enter value to search:15
15 not found
```

```
1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Searching
10.Sorting
11.Display/Traverse
12.Exit
Choice: 10
Sorting Completed

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Searching
10.Sorting
11.Display/Traverse
12.Exit
Choice: 11
10 -> 20 -> 30 -> None

1.Create Linked List
2.Insert Beginning
3.Insert End
4.Insert at Index
5.Delete by Value
6.Delete First Node
7.Delete Last Node
8.Count Number of Nodes
9.Searching
10.Sorting
11.Display/Traverse
12.Exit
Choice: 12
>>>
```

WEEK – 6

PROGRAM-6:

AIM: Write a python program to implement a Double Linked List with the below operations.

Create Linked List, Insert Beginning, Insert End, Insert at Index, Delete by Value, Delete First Node, Delete Last Node, Count number of nodes, Display or Traverse and Exit

CODE:

```
class Node:
    def __init__(self,data):
        self.data = data
        self.next = None
        self.prev = None
class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
    def create(self):
        n = int(input(("Enter number of elements:")))
        for i in range(n):
            x = int(input(("Enter value:")))
            new = Node(x)
            if self.head is None:
                self.head = new
                self.tail = new
            else:
                self.tail.next = new
                new.prev = self.tail
                self.tail = new
    def insert_begin(self,data):
        new = Node(data)
        if self.head is None:
            self.head = new
            self.tail = new
        else:
            self.head.prev = new
            new.next = self.head
            self.head = new
    def insert_end(self,data):
        new = Node(data)
        if self.head is None:
            self.head = new
            self.tail = new
        else:
            self.tail.next = new
            new.prev = self.tail
            self.tail = new
```

```
    def insert_index(self,index,data):
        if index < 0:
            print("Invalid Index")
            return
        if index == 0:
            self.index_begin(data)
            return
        n = self.count()
        if index > n:
            print("Invail Inadex")
            return
        new = Node(data)
        if index == n:
            self.index_end(data)
            return
        temp = self.head
        for i in range(index - 1):
            temp = temp.next
        new.next = temp.next
        new.prev = temp
        temp.next.prev = new
        temp.next = new
        print("Insertion Completed")
    def deleteBeg(self):
        if self.head is None:
            print("Can't perform delete operation")
        elif self.head.next is None:
            self.head = None
            self.tail = None
            print("Value Deleted")
        else:
            self.head = self.head.next
            self.head.prev = None
            print("Value Deleted")
    def deleteEnd(self):
        if self.head is None:
            print("Can't perform delete operation")
        elif self.head.next is None:
            self.head = None
            self.tail = None
            print("Value Deleted")
        else:
            self.tail = self.tail.prev
            self.tail.next = None
            print("Value Deleted")

    def deleteIndex(self,index):
        if self.head is None:
            print("Can't perform delete operation")
        elif index < 0 or index >= self.count():
            print("Invalid Index")
        elif index == 0:
            self.deleteBeg()
        elif index == self.count()-1:
            self.deleteEnd()
        else:
            temp = self.head
            for i in range(index):
                temp = temp.next
            temp.next.prev = temp.prev
            temp.prev.next = temp.next
            print("Value Deleted")
    def count(self):
        count = 0
        temp = self.head
        while temp:
            count += 1
            temp = temp.next
        return count
    def displayFwd(self):
        if self.head is None:
            print("No Data")
            return
        temp = self.head
        while temp:
            print(temp.data,end = "<->")
            temp = temp.next
        print("None")
    def displayBwd(self):
        if self.head is None:
            print("No Data")
            return
        temp = self.tail
        while temp:
            print(temp.data,end = "<->")
            temp = temp.prev
        print("None")
dll = DoublyLinkedList()
while True:
    print("\n1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
```

```
print("5. Delete Beginning")
print("6. Delete End")
print("7. Delete at Index")
print("8. Display Forward")
print("9. Display Backward")
print("10. Count")
print("11. EXIT")
ch = int(input("Enter your choice: "))
if ch == 1:
    dll.create()
elif ch == 2:
    data = int(input("Enter value: "))
    dll.insert_begin(data)
elif ch == 3:
    data = int(input("Enter value: "))
    dll.insert_end(data)
elif ch == 4:
    index = int(input("Enter index: "))
    data = int(input("Enter value: "))
    dll.insert_index(index, data)
elif ch == 5:
    dll.deleteBeg()
elif ch == 6:
    dll.deleteEnd()
elif ch == 7:
    index = int(input("Enter index: "))
    dll.deleteIndex(index)
elif ch == 8:
    dll.displayFwd()
elif ch == 9:
    dll.displayBwd()
elif ch == 10:
    print("Number of elements:", dll.count())
elif ch == 11:
    print("Program Ended")
    break
else:
    print("Invalid option")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/1a.py ============

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete Beginning
6. Delete End
7. Delete at Index
8. Display Forward
9. Display Backward
10. Count
11. EXIT
Enter your choice: 1
Enter number of elements:3
Enter value:10
Enter value:20
Enter value:30

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete Beginning
6. Delete End
7. Delete at Index
8. Display Forward
9. Display Backward
10. Count
11. EXIT
Enter your choice: 4
Enter index: 2
Enter value: 40
Insertion Completed
```

```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete Beginning
6. Delete End
7. Delete at Index
8. Display Forward
9. Display Backward
10. Count
11. EXIT
Enter your choice: 7
Enter index: 2
Value Deleted

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete Beginning
6. Delete End
7. Delete at Index
8. Display Forward
9. Display Backward
10. Count
11. EXIT
Enter your choice: 8
10<->20<->30<->None

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete Beginning
6. Delete End
7. Delete at Index
8. Display Forward
9. Display Backward
10. Count
11. EXIT
Enter your choice: 9
30<->20<->10<->None
```

```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete Beginning
6. Delete End
7. Delete at Index
8. Display Forward
9. Display Backward
10. Count
11. EXIT
Enter your choice: 10
Number of elements: 3

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete Beginning
6. Delete End
7. Delete at Index
8. Display Forward
9. Display Backward
10. Count
11. EXIT
Enter your choice: 11
Program Ended
>>>
```

WEEK – 7

PROGRAM-7(A):

AIM: Write a python program to implement stack operations such as push, pop, peek, display using an array.

CODE:

```
class StackEx:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1
    # Push operation
    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")
    # Pop operation
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")
    # Peek operation
    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])
    # Display operation
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            for i in range(self.top, -1, -1):
                print(self.stack[i])
# Create stack
size = int(input("Enter the size of the stack: "))
s = StackEx(size)
while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/
Enter the size of the stack: 3

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 10
10 pushed into the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 20
20 pushed into the stack
```

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
20 popped from the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 10

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the stack are:
10

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
>>>
```

PROGRAM-7(B):

AIM: Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Stack:
    def __init__(self):
        self.top = None
    # Push operation
    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "pushed into stack")
    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            print(temp.data, "popped from stack")
            self.top = self.top.next
    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)
    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next
```

```
s = Stack()
while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the element: "))
        s.push(data)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 10
10 pushed into stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 20
20 pushed into stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
20 popped from stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 10

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
10

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
>>>
```

WEEK – 8

PROGRAM-8(A):

AIM: Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

CODE:

```
class QueueEx:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
    # Enqueue operation
    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")
    # Dequeue operation
    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")
            # Reset queue when it becomes empty
            if self.front > self.rear:
                self.front = -1
                self.rear = -1
    # Peek operation
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])
    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])
```

```
size = int(input("Enter the size of the queue: "))
q = QueueEx(size)
while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/
Enter the size of the queue: 3

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 10
10 inserted into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 20
20 inserted into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
10 deleted from the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 20

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
20

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
>>>
```

PROGRAM-8(B):

AIM: Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

CODE:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
    # Enqueue operation
    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
    # Dequeue operation
    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        # Only one element is present
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        # More than one element
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)
while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.display()
    elif choice == 4:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
    ========== RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/
    Enter the size of the queue: 3

    ----- CIRCULAR QUEUE MENU -----
    1. Enqueue
    2. Dequeue
    3. Display
    4. Exit
    Enter your choice: 1
    Enter the element to enqueue: 10
    10 inserted into the queue

    ----- CIRCULAR QUEUE MENU -----
    1. Enqueue
    2. Dequeue
    3. Display
    4. Exit
    Enter your choice: 1
    Enter the element to enqueue: 20
    20 inserted into the queue
```

```
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
10 deleted from the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
The elements of the queue are:
20

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Program terminated.
>>>
```

WEEK – 9

PROGRAM-9:

AIM: Write a python program to implement a Binary Search Tree (BST) and perform the following operations.

Insertion, Deletion, Tree Traversals – in-order, pre-order and post-order, Count Leaf Nodes, Count Internal Nodes and Height of the Tree

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
# INSERTION
def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    return root
# INORDER: Left, Root, Right
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
# PREORDER: Root, Left, Right
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
# POSTORDER: Left, Right, Root
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
# COUNT LEAF NODES
def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)
```

```
# COUNT INTERNAL NODES
def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1
# HEIGHT OF TREE
def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1
# DELETION
def delete(root, x):
    if root is None:
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            temp = root.right
            while temp.left is not None:
                temp = temp.left
            root.data = temp.data
            root.right = delete(root.right, temp.data)

    return root
# MAIN PROGRAM
root = None
while True:
    print("\n***** BINARY SEARCH TREE *****")
    print("1. INSERT")
    print("2. DELETE")
    print("3. INORDER TRAVERSAL")
    print("4. PREORDER TRAVERSAL")
    print("5. POSTORDER TRAVERSAL")
    print("6. COUNT LEAF NODES")
    print("7. COUNT INTERNAL NODES")
    print("8. HEIGHT OF TREE")
    print("9. EXIT")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)
        print("Element inserted successfully.")
    elif choice == 2:
        x = int(input("Enter value to delete: "))
        root = delete(root, x)
        print("Deletion operation completed.")
    elif choice == 3:
        print("Inorder:", end=" ")
        inorder(root)
        print()
    elif choice == 4:
        print("Preorder:", end=" ")
        preorder(root)
        print()
    elif choice == 5:
        print("Postorder:", end=" ")
        postorder(root)
        print()
    elif choice == 6:
        print("Leaf nodes:", count_leaves(root))
    elif choice == 7:
        print("Internal nodes:", count_internal(root))
    elif choice == 8:
        print("Height of tree:", height(root))
    elif choice == 9:
        print("Program terminated.")
        break
    else:
        print("Invalid choice.")
```

OUTPUT:

```
>>>
========== RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/

***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 1
Enter value: 10
Element inserted successfully.

***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 1
Enter value: 20
Element inserted successfully.

***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 1
Enter value: 30
Element inserted successfully.
```

```
***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 1
Enter value: 40
Element inserted successfully.

***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 2
Enter value to delete: 40
Deletion operation completed.

***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 3
Inorder: 10 20 30
```

```
***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 6
Leaf nodes: 1

***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 7
Internal nodes: 2

***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 8
Height of tree: 2

***** BINARY SEARCH TREE *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 9
Program terminated.
>>>
```

WEEK – 10

PROGRAM-10:

AIM: Write a menu driven python program to implement an AVL Tree and perform the following operations.

Insertion, Deletion, Tree Traversals, Count Leaf Nodes, Count Internal Nodes and Height of the Tree

CODE:

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0
# HEIGHT
def height(root):
    if root is None:
        return -1
    return root.h
# LL ROTATION
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y
# RR ROTATION
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y
# LR ROTATION
def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)
# RL ROTATION
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)
```

```
# INSERTION
def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)
    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")

    root.h = max(height(root.left), height(root.right)) + 1
    return root
# INORDER TRAVERSAL
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
# PREORDER TRAVERSAL
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
# POSTORDER TRAVERSAL
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
```

```
# COUNT LEAF NODES
def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)
# COUNT INTERNAL NODES
def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1
```

```
# DELETE
def delete(root, x):
    if root is None:
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        else:
            successor = root.right
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    # UPDATE HEIGHT
    root.h = max(height(root.left), height(root.right)) + 1
    # BALANCE FACTOR
    balance = height(root.left) - height(root.right)
    # LL CASE
    if balance > 1 and height(root.left.left) >= height(root.left.right):
        return LL(root)
    # LR CASE
    if balance > 1:
        return LR(root)
    # RR CASE
    if balance < -1 and height(root.right.right) >= height(root.right.left):
        return RR(root)
    # RL CASE
    if balance < -1:
        return RL(root)
    return root
# MAIN PROGRAM
root = None
while True:
    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. DELETE")
    print("3. INORDER TRAVERSAL")
    print("4. PREORDER TRAVERSAL")
    print("5. POSTORDER TRAVERSAL")
    print("6. COUNT LEAF NODES")

    print("7. COUNT INTERNAL NODES")
    print("8. HEIGHT OF TREE")
    print("9. EXIT")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
        print("Element inserted successfully.")
    elif choice == 2:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
        print("Deletion completed.")
    elif choice == 3:
        print("Inorder Traversal:", end=" ")
        inorder(root)
        print()
    elif choice == 4:
        print("Preorder Traversal:", end=" ")
        preorder(root)
        print()
    elif choice == 5:
        print("Postorder Traversal:", end=" ")
        postorder(root)
        print()
    elif choice == 6:
        print("Leaf Nodes:", count_leaves(root))
    elif choice == 7:
        print("Internal Nodes:", count_internal(root))
    elif choice == 8:
        print("Height of Tree:", height(root))
    elif choice == 9:
        print("Program terminated.")
        break
    else:
        print("Invalid choice!")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/

***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 1
Enter the data to be inserted: 10
Element inserted successfully.
```

```
***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 1
Enter the data to be inserted: 20
Element inserted successfully.

***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 1
Enter the data to be inserted: 30
Element inserted successfully.

***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 1
Enter the data to be inserted: 40
Element inserted successfully.
```

```
***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 2
Enter the data to be deleted: 40
Deletion completed.

***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 5
Postorder Traversal: 10 30 20

***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 6
Leaf Nodes: 2
```

```
***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 7
Internal Nodes: 1
```

```
***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 8
Height of Tree: 1

***** AVL TREE MENU *****
1. INSERT
2. DELETE
3. INORDER TRAVERSAL
4. PREORDER TRAVERSAL
5. POSTORDER TRAVERSAL
6. COUNT LEAF NODES
7. COUNT INTERNAL NODES
8. HEIGHT OF TREE
9. EXIT
Enter your choice: 9
Program terminated.
>>>
```

WEEK – 11

PROGRAM-11:

AIM: Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

Insertion of an element into the Heap, Deletion of the root element from the Heap, Display the array representation of the Heap and Exit the program.

CODE:

```
# MAX HEAP INSERTION
def insert_max(heap, value):
    heap.append(value)
    i = len(heap) - 1

    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break
# MAX HEAP DELETION
def delete_max(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()
    n = len(heap)
    i = 0
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and heap[left] > heap[largest]:
            largest = left
        if right < n and heap[right] > heap[largest]:
            largest = right
        if largest != i:
            heap[i], heap[largest] = heap[largest], heap[i]
            i = largest
        else:
            break
# MIN HEAP INSERTION
def insert_min(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] < heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break
```

```
# MIN HEAP DELETION
def delete_min(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()
    n = len(heap)
    i = 0
    while True:
        smallest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and heap[left] < heap[smallest]:
            smallest = left
        if right < n and heap[right] < heap[smallest]:
            smallest = right
        if smallest != i:
            heap[i], heap[smallest] = heap[smallest], heap[i]
            i = smallest
        else:
            break
# DISPLAY
def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)
# MAIN PROGRAM
max_heap = []
min_heap = []
while True:
    print("\n***** BINARY HEAP MENU *****")
    print("1. Insert into Max Heap")
    print("2. Delete from Max Heap")
    print("3. Display Max Heap")
    print("4. Insert into Min Heap")
    print("5. Delete from Min Heap")
    print("6. Display Min Heap")
    print("7. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value: "))
        insert_max(max_heap, value)
        print("Element inserted successfully")
    elif choice == 2:
        delete_max(max_heap)
    elif choice == 3:
        display(max_heap)
    elif choice == 4:
        value = int(input("Enter value: "))
        insert_min(min_heap, value)
        print("Element inserted successfully")
    elif choice == 5:
        delete_min(min_heap)
    elif choice == 6:
        display(min_heap)
    elif choice == 7:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 1
Enter value: 10
Element inserted successfully

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 1
Enter value: 20
Element inserted successfully
```

```
***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 1
Enter value: 30
Element inserted successfully

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 2
Deleted element: 30

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 30
Invalid choice

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 3
Heap elements: [20, 10]
```

```
***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 4
Enter value: 10
Element inserted successfully

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 4
Enter value: 20
Element inserted successfully

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 4
Enter value: 30
Element inserted successfully

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 5
Deleted element: 10
```

```
***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 6
Heap elements: [20, 30]

***** BINARY HEAP MENU *****
1. Insert into Max Heap
2. Delete from Max Heap
3. Display Max Heap
4. Insert into Min Heap
5. Delete from Min Heap
6. Display Min Heap
7. Exit
Enter your choice: 7
Program terminated
```

WEEK – 12

PROGRAM-12:

AIM: Write a program to implement the following three collision resolution techniques in a Hash Table:

Separate Chaining, Linear Probing and Quadratic Probing

CODE:

```
# SEPARATE CHAINING
def chaining(table, value, size):
    index = value % size
    table[index].append(value)
# LINEAR PROBING
def linear(table, value, size):
    index = value % size
    i = 0
    while i < size:
        pos = (index + i) % size
        if table[pos] is None:
            table[pos] = value
            return
        i += 1
    print("Hash Table is full")
# QUADRATIC PROBING
def quadratic(table, value, size):
    index = value % size
    i = 0
    while i < size:
        pos = (index + i * i) % size
        if table[pos] is None:
            table[pos] = value
            return
        i += 1
    print("Hash Table is full")
# DISPLAY
def display(table):
    for i in range(len(table)):
        print(i, ":", table[i])
# MAIN PROGRAM
size = int(input("Enter hash table size: "))
chain_table = [[] for i in range(size)]
linear_table = [None] * size
quad_table = [None] * size
while True:
    print("\n***** HASH TABLE MENU *****")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Display All")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        value = int(input("Enter value: "))
        chaining(chain_table, value, size)
        print("Element inserted successfully")
    elif choice == 2:
        value = int(input("Enter value: "))
        linear(linear_table, value, size)
```

```
    elif choice == 3:
        value = int(input("Enter value: "))
        quadratic(quad_table, value, size)
    elif choice == 4:
        print("\nSeparate Chaining:")
        display(chain_table)
        print("\nLinear Probing:")
        display(linear_table)
        print("\nQuadratic Probing:")
        display(quad_table)
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/PUTTA YASASVI/OneDrive/Desktop/
Enter hash table size: 3

***** HASH TABLE MENU *****
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 1
Enter value: 10
Element inserted successfully

***** HASH TABLE MENU *****
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 1
Enter value: 20
Element inserted successfully

***** HASH TABLE MENU *****
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 2
Enter value: 30

***** HASH TABLE MENU *****
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 3
Enter value: 40

***** HASH TABLE MENU *****
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 4

Separate Chaining:
0 : []
1 : [10]
2 : [20]

Linear Probing:
0 : 30
1 : None
2 : None

Quadratic Probing:
0 : None
1 : 40
2 : None

***** HASH TABLE MENU *****
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 5
Program terminated
>>>
```